

TECHNICAL DOCUMENTATION

Event Management System

A full-stack platform for creating, managing, and booking events — built with the MERN stack, real-time capabilities, and payment integration.

10
Weeks

7
Phases

Full
Stack

TECH STACK

React.js

Node.js

MongoDB

JWT

Stripe

Socket.io

Cloudinary

QR Codes

TABLE OF CONTENTS

01	Project Setup	Week 1 — Foundation & scaffolding
02	Authentication	Week 2 — Security & access control
03	Event Module	Weeks 3–4 — Core feature
04	Ticket & Booking	Weeks 5–6 — Commerce layer
05	Dashboard	Week 7 — Analytics & reporting
06	Real-time Features	Week 8 — Live updates & notifications
07	Testing & Deployment	Weeks 9–10 — QA & production

01

Project Setup

Week 1 — Foundation & scaffolding

Establish the monorepo structure, configure tooling, and connect all services. This phase lays the groundwork that every later phase depends on.

DELIVERABLES

Initialize React + Node.js Projects

Use Vite for React frontend. Create Express app for backend. Set up separate /client and /server directories in a monorepo. Install ESLint + Prettier across both.

Setup MongoDB Connection

Use Mongoose ODM. Configure connection string in .env. Implement connection retry logic. Create base model patterns for timestamps and soft deletes.

Configure Environment Variables

Separate .env files for dev, test, prod. Required vars: MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, CLOUDINARY_*, STRIPE_SECRET_KEY, SMTP_*, CLIENT_URL.

Basic Project Structure

Backend: /routes, /controllers, /models, /middleware, /services, /utils. Frontend: /pages, /components, /hooks, /context, /api, /assets.

02

Authentication

Week 2 — Security & access control

Full auth system with JWT access/refresh tokens, role-based middleware for Admin / Organizer / Attendee, and a complete password reset flow via email OTP.

DELIVERABLES

Register / Login / Logout

Password hashing with bcrypt (salt rounds: 12). Email verification on register. Refresh token stored in httpOnly cookie. Access token in memory / Authorization header.

JWT Implementation

Access token: 15min expiry. Refresh token: 7d expiry stored in DB. Token rotation on every refresh. Blacklist invalidated tokens in Redis or MongoDB set.

Role-based Middleware

Roles: admin, organizer, attendee. Middleware chain: verifyToken -> checkRole(roles[]). Guards applied per-route. Admin can access all routes; organizer manages own events only.

Password Reset Flow

Request reset -> generate 6-digit OTP -> store hashed OTP with 10min TTL -> send email -> verify OTP -> update password -> invalidate all sessions.

API ENDPOINTS

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

POST /api/auth/refresh-token

POST /api/auth/forgot-password

POST /api/auth/reset-password

USER SCHEMA

name String	email String, unique
password String (hashed)	role Enum
isVerified Boolean	refreshToken String
resetOTP String	otpExpiry Date

03**Event Module**

Weeks 3–4 — Core feature

The core product. Full CRUD for events with Cloudinary image hosting, a MongoDB text-indexed search API, filter/sort/paginate, and a polished listing UI in React.

DELIVERABLES**Event CRUD Operations**

Create, Read, Update, Delete events. Organizer can only manage their own. Soft delete with isDeleted flag. Event statuses: draft, published, cancelled, completed.

Image Upload (Cloudinary)

Multer middleware for multipart parsing. Upload to Cloudinary with auto-transformation (resize to 1200x630). Store secure_url and public_id. Delete old image on update.

Search & Filter API

MongoDB text index on title, description, tags. Query params: q, category, city, date_from, date_to, price_min, price_max, sort (date|price|popular). Cursor-based pagination.

Event Listing UI

Card grid with skeleton loaders. Filter sidebar (category, price range, date picker). Infinite scroll or paginated view. Event detail page with tabbed sections: info, tickets, map.

API ENDPOINTS

GET	/api/events?q=&category;=&city;=
GET	/api/events/:id
POST	/api/events (organizer)
PUT	/api/events/:id
DEL	/api/events/:id
POST	/api/events/:id/upload

EVENT SCHEMA

title String	description String
category Enum	date Date
location Object	organizer ObjectId ref User
image { url, publicId }	status Enum
tags [String]	capacity Number

04

Ticket & Booking

Weeks 5–6 — Commerce layer

Multi-tier ticketing with inventory management, Stripe Checkout integration, webhook handling, and QR code generation for digital admission.

DELIVERABLES

Ticket Type Management

Per-event ticket types: General, VIP, Early Bird. Each type has: name, price, quantity, quantitySold, saleStart, saleEnd. Organizer manages types from dashboard.

Booking System

Atomic quantity decrement with MongoDB transactions. Booking statuses: pending, confirmed, cancelled, refunded. 15-minute lock on selected tickets before payment confirmation.

Payment Integration (Stripe)

Create PaymentIntent on booking. Stripe Checkout session for UI. Webhook: payment_intent.succeeded -> confirm booking, payment_intent.payment_failed -> release lock. Store charge ID.

QR Code Generation

Generate unique QR per ticket using qrcode npm package. Encode: bookingId + ticketId + HMAC signature. Send as base64 in booking confirmation email. Scanner validates HMAC.

API ENDPOINTS

POST /api/bookings/create

GET /api/bookings/my

POST /api/bookings/:id/cancel

POST /api/payments/create-intent

POST /api/payments/webhook

GET /api/tickets/:id/qr

05**Dashboard**

Week 7 — Analytics & reporting

Role-specific dashboards for admins and organizers. Revenue charts, booking trends, attendee demographics, and exportable PDF/CSV reports.

DELIVERABLES**Admin Dashboard**

KPIs: total users, total events, total revenue, active bookings. User management table with role change and ban. Approve/reject organizer requests. Platform-wide revenue charts.

Organizer Dashboard

My Events list with status badges. Per-event stats: tickets sold, revenue, capacity %. Attendee list per event. Edit/cancel event. Revenue timeline chart (Recharts).

Analytics & Charts

Recharts: line chart for bookings over time, bar chart for revenue by category, pie chart for ticket types. Date range filter. Aggregate queries via MongoDB aggregation pipeline.

Reports Generation

Export attendee list as CSV (papaparse). PDF summary report with event stats using pdfkit or puppeteer. Download triggered from dashboard. Emailed report option for organizers.

06

Real-time Features

Week 8 — Live updates & notifications

Socket.io integration for live ticket availability and in-app notifications. Nodemailer for transactional emails. Optional Twilio SMS alerts for booking confirmations.

DELIVERABLES

Socket.io Setup

Attach Socket.io to existing HTTP server. Authenticate socket connections with JWT middleware. Join rooms by event ID. Emit ticket availability updates on every booking transaction.

Live Notifications

In-app notification bell. Notification model stored in DB. Events: booking_confirmed, event_updated, event_cancelled, new_follower. Unread count badge. Mark all as read.

Email Notifications

Nodemailer with Gmail OAuth2 or SMTP. Templates: booking confirmation (with QR), event reminder (24h before), password reset OTP, organizer revenue report. HTML email with inline CSS.

SMS Alerts (Optional)

Twilio SMS API for booking confirmation and event day reminders. Opt-in field on user profile. Feature-flagged behind ENABLE_SMS env var. Rate limited to prevent abuse.

07

Testing & Deployment

Weeks 9–10 — QA & production

API testing with Postman collections, Jest unit tests, React Testing Library for components, and CI/CD deployment to Render (backend) and Vercel (frontend).

DELIVERABLES

API Testing (Postman)

Postman collection with environment variables for dev/prod. Pre-request scripts for auth token injection. Automated test scripts on response: status codes, schema validation, error cases.

Frontend Testing

Jest + React Testing Library. Unit tests for utility functions. Component tests: render, user interaction, async loading states. MSW for API mocking. Coverage threshold: 70%.

Bug Fixes & Hardening

Helmet.js for HTTP headers. Rate limiting with express-rate-limit. Input sanitization with express-validator. CORS configured for production domain only. Morgan logging in production.

Deploy to Production

Backend to Render.com (auto-deploy from main). Frontend to Vercel with env vars. MongoDB Atlas for DB. GitHub Actions CI: lint -> test -> build -> deploy. Custom domain + SSL.

DEPLOYMENT STACK

Frontend Vercel	Backend Render.com
Database MongoDB Atlas	Images Cloudinary CDN
CI/CD GitHub Actions	Monitoring Render Logs